Jetpack Compose

Muhammad Naveed

November 17, 2025

Table of Contents

State & Recomposition

The core of dynamic UI. What state is, how it works, and how to manage it effectively.

CompositionLocal

How to pass data deep into your UI tree without "props drilling".

Modifiers

The building blocks for styling, decorating, and adding behavior to your composables.

Chapter 22: State & Recomposition

The core of dynamic UI

What is State?

"A value that can change over time" that triggers UI updates when modified.

- ✓ **Remembered**
  State values persist across recompositions.
- ✓ **Observable**
  Changes automatically trigger UI updates.

Example: Instagram Like Button

Flow:

- ✓ **Initial State:**

  `isLiked = false`

  `likeCount = 63`

- ✓ **Action:**

  User taps heart button.

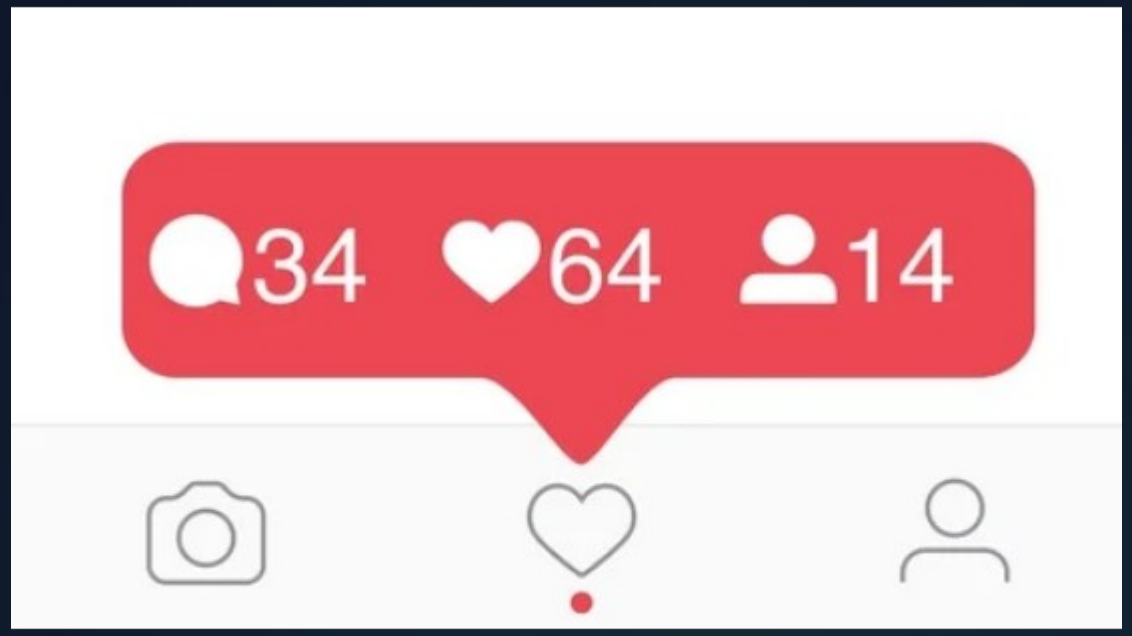
- ✓ **State Change:**

  `isLiked = true`

  `likeCount = 64`

- ✓ **Recomposition:**

  Heart turns red, count displays 64.



Recomposition Explained

What is Recomposition?

The automatic re-execution of composable functions when their state changes.

Intelligent Recomposition

Only functions that **READ** the changed state are recomposed, not the entire UI tree.

Analogy: A smart home only turns on lights in rooms where motion is detected, not the entire house.

Three Ways to Declare State

1

Direct Value Access

```
var textState = remember {
mutableStateOf("") }

textState.value = "New Text"

val text = textState.value
```

2

Property Delegates (Best)

```
var textState by remember {
mutableStateOf("") }

textState = "New Text"

val text = textState
```

3

Destructuring

```
var (text, setText) =
remember {
mutableStateOf("") }

setText("New Text")

val value = text
```

Unidirectional Data Flow (UDF)

The Golden Rule

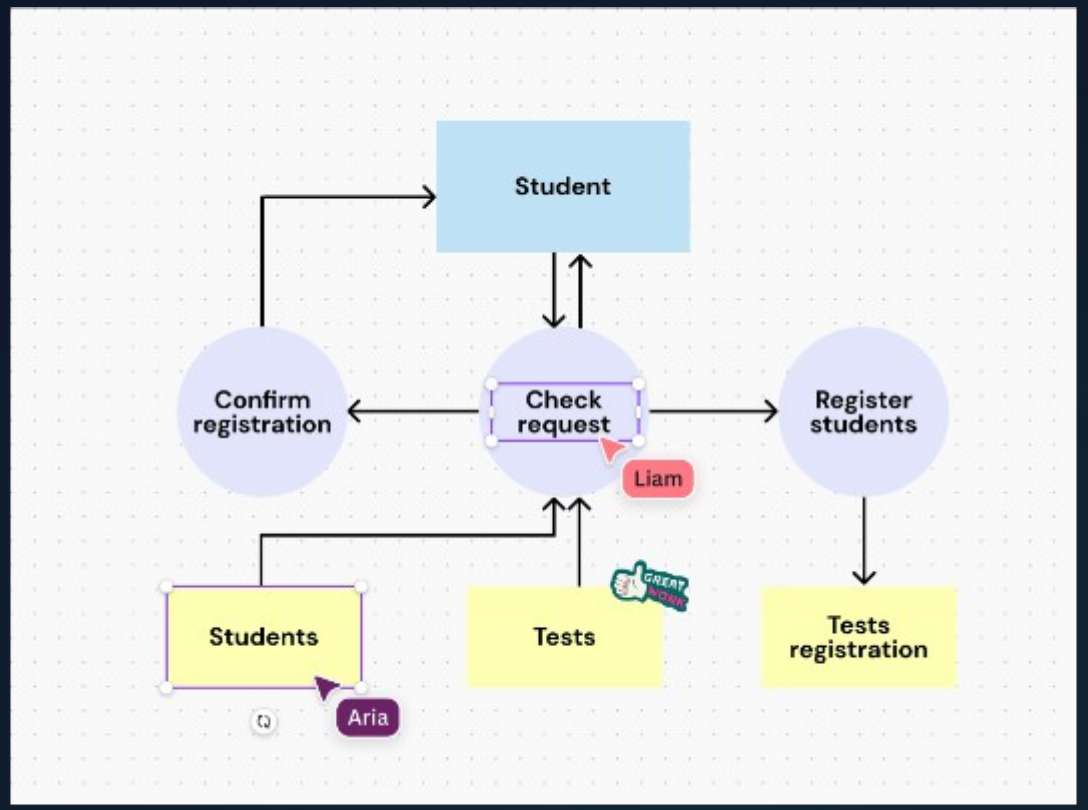
A pattern where data flows in one direction, making the app more predictable and easier to debug.

Data Flows DOWN

Parents pass state down to children.

Events Flow UP

Children expose handlers (lambdas) that notify the parent to update its state.



UDF Example: Shopping Cart

- **Parent Screen** holds cartItems state.
- **Passes State DOWN** (item details) and onAddToCart handler DOWN to ProductCard.
- **User clicks "Add"** in the ProductCard. ProductCard calls the onAddToCart() handler **UP**.
- **Parent Screen** updates cartItems state.
- **Recomposition** automatically updates the UI.

State Hoisting

What is State Hoisting?

Moving state UP from a child to a parent or higher ancestor.

How High to Hoist?

The **Lowest Common Ancestor (LCA)** is the lowest parent in the UI tree that owns all the children that need to read or write the state.

Why Hoist?

- ✓ **Reusable:** Makes child components stateless and highly reusable.
- ✓ **Shareable:** Allows state to be shared with sibling components.
- ✓ **Testable:** Easier to test stateless components and state logic separately.

Problem: Configuration Changes

Rotating your phone destroys and recreates the entire Activity. How do you keep your state?

Function	Survives Recomposition	Survives Rotation
remember { ... }	✅	❌
rememberSaveable { ... }	✅	✅

Use rememberSaveable for things like text in an email draft!

Chapter 23: CompositionLocal

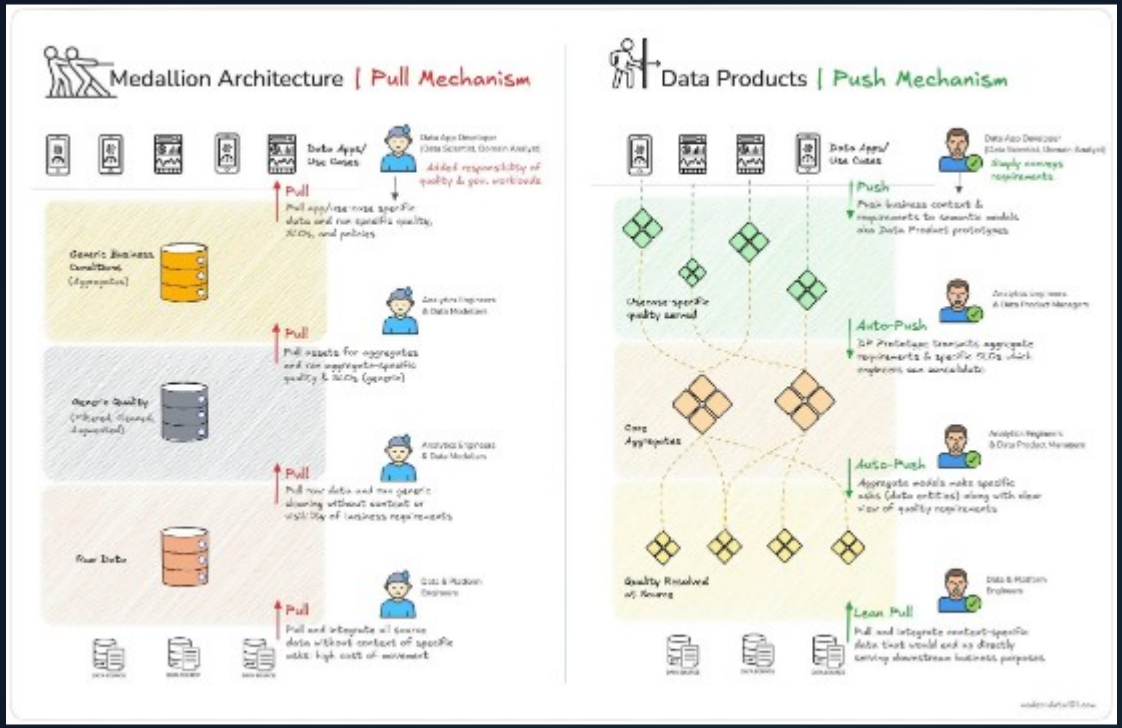
Escaping "Props Drilling"

The "Props Drilling" Problem

Passing state (like theme data) through multiple layers of components that don't actually use it.

```
Parent(data)
  -> Middle(data) // Doesn't use data!
    -> Layer(data) // Doesn't use data!
      -> DeepChild(data) // Finally uses data!
```

This makes intermediate components like Middle and Layer cluttered and tightly coupled to data they don't care about.



The Solution: CompositionLocal

Provides a way to make state available to all descendants WITHOUT passing it as a parameter through every layer.

1. Create: Define a "key" for your data.

```
val LocalColor = compositionLocalOf { Color.Red }
```

2. Provide: Set the value at a parent level.

```
CompositionLocalProvider(LocalColor provides Color.Blue) { ... }
```

3. Access: Any descendant can read the value directly.

```
val color = LocalColor.current
```

Example: Theme Management

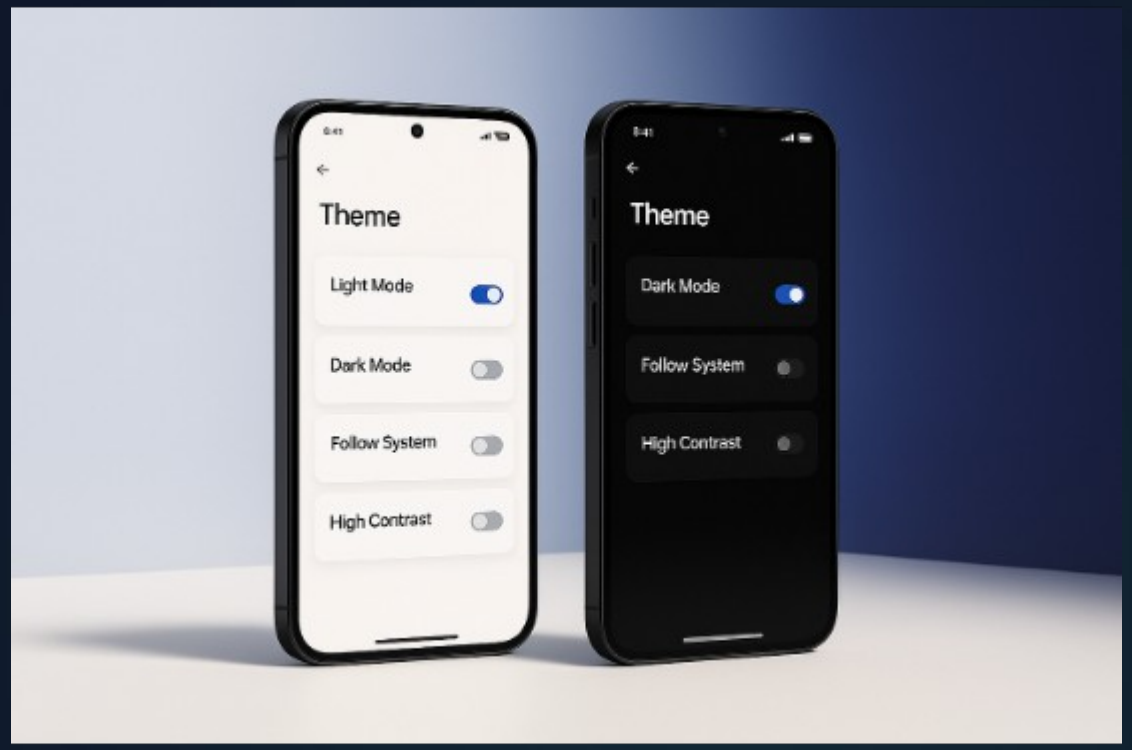
Without CompositionLocal

You would have to pass themeColors as a parameter to **every single component** in your app. Unmanageable!

With CompositionLocal

MaterialTheme provides LocalColors, LocalTypography, etc. at the root of your app.

Any component, no matter how deep, can directly access LocalColors.current to get the right theme color.



Two Types of CompositionLocal

1. compositionLocalOf()

- ✓ **Use For:** Frequent Changes
- ✓ **Examples:** User input state, animation values, scroll positions.
- ✓ **Benefit:** Smartly recomposes **only** the components that read the value.

2. staticCompositionLocalOf()

- ✓ **Use For:** Rare Changes
- ✓ **Examples:** Theme colors (changes once), app configuration, font families.
- ✓ **Tradeoff:** Recomposes the entire branch below the provider (less efficient for frequent changes, but fine if it's rare).

Chapter 26: Modifiers

Building blocks of appearance and behavior

What are Modifiers?

Configuration objects that modify the appearance and behavior of composables. They are standard, chainable, and powerful.

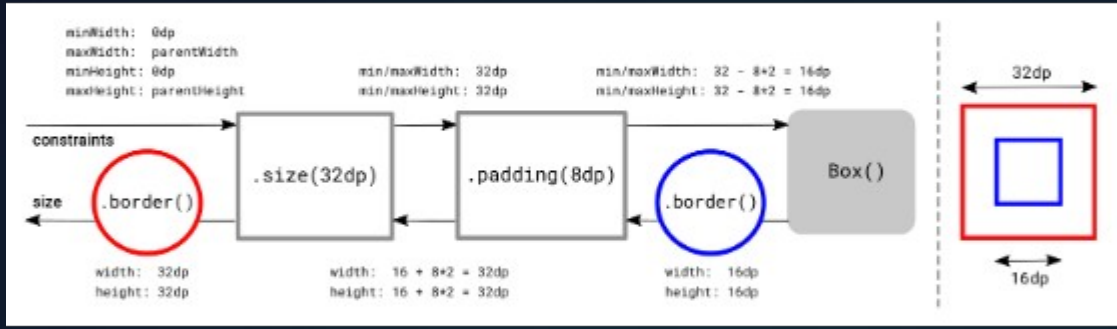
- **Size & Layout:** .width(), .height(), .fillMaxSize(), .padding()
- **Visual Style:** .background(), .border(), .shadow(), .clip()
- **Interactions:** .clickable(), .draggable(), .scrollable()
- **Animations:** .rotate(), .scale(), .alpha()

CRITICAL: Modifier Order Matters!

Modifiers are applied sequentially. Think of them as wrapping the composable, one after another.

Border Outside Padding

```
Modifier
    .padding(16.dp)
    .border(2.dp, Color.Black)
```



Border Inside Padding

```
Modifier
    .border(2.dp, Color.Black)
    .padding(16.dp)
```

Best Practice: Support Modifiers

Make Your Composables Flexible

Your custom composables should always accept a modifier as the first optional parameter and apply it to the root element.

This lets consumers of your composable change its padding, size, background, etc., making it truly reusable.

```
@Composable
fun CustomButton(
    text: String,
    modifier: Modifier = Modifier, // 1. Add as parameter
    onClick: () -> Unit
) {
    Button(
        onClick = onClick,
        modifier = modifier // 2. Apply to root
    ) {
        Text(text)
    }
}
```

Key Takeaways

- **State Drives UI:** When state changes, UI recomposes. Use `rememberSaveable` for configuration changes.
- **Unidirectional Flow:** Data flows down, events flow up. This creates a predictable and debuggable app.
- **Hoist State:** Move state to the **Lowest Common Ancestor** to make components stateless and reusable.
- **CompositionLocal:** Solves "props drilling" for passing "global-ish" data (like themes) deep into the UI tree.
- **Modifiers are Powerful:** Chain them to build complex UI. **Order is critical!**

Questions?

Thank you!